

CIA TRIAD + EXTENDED PROPERTIES

- **Confidentiality:** Only authorised users can READ
- **Integrity:** Data cannot be changed without permission
- **Availability:** System accessible when needed (DoS)
- **Authenticity:** Verify the entity is genuinely who it is
- **Accountability:** All actions traceable to a person
- **Non-repudiation:** Cannot deny having performed an action

Attack targets:

DoS/DDoS -> Availability

MitM -> Confidentiality + Integrity

ACCESS CONTROL — DAC / MAC / RBAC / DAC Matrix Rules (Crib Sheet p.1):

R2 grant {a} to S, X

"owner" in A[S0,X]

R3 delete a from S, X

"owner" OR "control" in A[S0,X]

R5 create object X

None required -> stores owner

R7 create subject S

None required -> stores control

R8 destroy subject S

"owner" in A[S0,S]

Policy comparison:

Model	Who decides	Flex	Best for
DIScretionary	Owner	High	Small systems
Mandatory	System	Low	High security
Role-based	Roles	Medium	Organisations
Attribute-based	Attributes	High	Complex systems

AUTHENTICATION — KNOW / HAVE / ARE / WHERE

KNOW

Password, PIN, security question

Risk: Phishing, guessing

HAVE

Smart card, OTP token, phone

Risk: Lost/stolen

ARE

Fingerprint, iris, face, gait

Risk: FAR/FRR errors

WHERE

Location, terminal, GPS

Risk: VPN/proxy bypass

FAR=false accept | FRR=false reject | CER=crossover (lower=better)

Attacks: Dictionary | Brute force | Rainbow table -> Salt

RSA — EVERY YEAR — HIGH PRIORITY

Key Generation (Crib Sheet p.3):

1 n = p × q (p, q both prime, p ≠ q)

2 phi(n) = (p-1)(q-1)

3 choose e: gcd(phi(n),e)=1, 1<e<phi(n)

4 find d: d*e mod phi(n) = 1 (use EEA)

Public: {e, n}

Private: {d, n}

Encrypt: c = M^e mod n

Decrypt: M = C^d mod n

Extended Euclidean Algorithm (EEA) — to find inverse

Phase 1 — Divide down (save each remainder):

phi(n) = q1 * e + r1 -> save: r1

e = q2 * r1 + r2 -> save: r2

r1 = q3 * r2 + r3 -> save: r3

... (continue until remainder = 0)

When remainder = 1, gcd = 1 -> invertible

Phase 2 — Back-substitute (work upwards):

1 = r_{n-1} - q * r_{n-2}

Replace each r using saved equation above

Until: 1 = A*e + B*phi(n)

Then: d = A mod phi(n) (may need to add phi(n) to get positive)

Verify: e * d mod phi(n) = 1 ✓

Repeated squaring — for decryption M=C^d mod n

4+32+16+1

1 2 3 4 5 6 7

0000 0001 0010 0011 0100 0101 0110 0111

CBC vs ECB (Crib Sheet p.3 diagram)

ECB — WEAK, never use for structured data

C_i = E(K, P_i) Same P_i -> same C_i. Pattern visible

CBC — SECURE (use this)

Encrypt: C_i = E(K, P_i XOR C_{i-1})

C_0 = IV (random, fresh per session)

Decrypt: P_i = D_K(C_i) XOR C_{i-1}

Error propagation: corrupt C_i ruins ONLY P_i and C_{i+1}

Worked Example (from 2025 Q3 / 2024 Q9):

P1=1101 P2=0110 P3=1001 | IV=1010 Key=0011

Block 1: 1101 XOR 1010 = 0111 -> 0111 XOR 0011 = 0100

Block 2: 0110 XOR 0100 = 0010 -> 0010 XOR 0011 = 0101

Block 3: 1001 XOR 0001 = 1000 -> 1000 XOR 0011 = 0111

Ciphertext: 0100 0001 0111

Hex <=> Binary (Crib Sheet p.4):

0 1 2 3 4 5 6 7

0000 0001 0010 0011 0100 0101 0110 0111

RC4 STREAM CIPHER (Crib Sheet p.3)

Step 1 Init: S[i]=i, T[i]=K[i] mod keylen

Step 2 KSA: j=(j+S[i]+T[i]) mod 256; swap S[i], S[j]

Step 3 PRGA: generate keystream byte; XOR with plaintext

Insecure: KSA invariance -> biased keystream -> known plaintext attack

Exam: show S-array state after each KSA iteration

SSO + KERBEROS (every year)

CIAAN properties under SSO:

● Confidentiality: ✓ Fewer credential transmissions

● Integrity: ✓ Tickets cryptographically signed

● Availability: ✗ SINGLE POINT OF FAILURE

● Authenticity: ✓ KDC verifies identity once

● Accountability: ✓ Central audit log

● Non-repudiation: ✓ Tied to verified identity

Kerberos 5 Steps:

1 User -> AS: username (plaintext), request TGT

2 AS -> User: TGT (enc. TGS key) + session key (enc. with AS key)

3 User -> TGS: TGT + Authenticator (timestamp, prev. session key)

4 TGS -> User: Service Ticket (enc. with service secret key)

5 User -> Service: Service Ticket + Authenticator -> access

Key: tickets encrypted with RECIPIENT's key. User cannot forge them.

WEB SECURITY — SQL / CSRF / XSS

HTTP vs HTTPS:

HTTP (port 80): plaintext. Password visible on network

HTTPS (port 443): TLS encrypt + CA cert + integrity MAC

SSL deprecated. TLS is current standard. Use HTTPS

SQL Injection

Malicious SQL in input field bypasses filters

Normal: SELECT * FROM users WHERE u='alice'

Attack: u = ' OR 1=1 -- (always true, bypasses filter)

Fix: Parameterised queries (prepared statements)

CSRF

Evil site fires request using victim's active session

Victim logged in to bank -> visits evil.com

Evil.com sends POST to bank using victim's session

Fix: CSRF tokens | SameSite cookie attribute

XSS

Injected JS runs in victim's browser in trusted context

Stored: script saved in DB, runs for every visitor

Reflected: script in URL, echoed back by server

Fix: HttpOnly cookies | CSP headers | sanitise inputs

DIGITAL FORENSICS — 4 STEPS

1 Seizure:

Isolate: DO NOT power on (RAM = volatile)

BIOS time crucial. Photos. Chain of custody log.

2 Acquisition:

Write blocker. Bit-for-bit image (FTImaging)

SHA-256 hash immediately. Never touch original.

3 Analysis:

Registry, browser history, deleted files, etc.

Work ONLY on verified copies. Whitelist comparison

4 Reporting:

Formal report. Hash values. Tool verification

Chain of custody. Admissible in court.

OSI MODEL — ATTACKS PER LAYER

Layer	Attack
7 App	SQL Inject, XSS, CSRF
4 Trans	SYN Spoofing (TCP 3-way exploit)
3 Net	IP Spoofing, routing attacks
2 Data	ARP Poisoning
1 Phys	Wiretapping, fibre tapping (PASSIVE)

Active:

Modifies/disrupts (DoS, MitM, spoofing)

Passive:

Only listens — no changes (fibre tapping)

HIGH PRIORITY:

RSA (every year, 10+ marks)

CBC/ECB (3/4 years)

Brute Force (3/4 years)

DAC Matrix (3/4 years)

SSO/Kerberos (3/4 years)

Show ALL working — partial marks awarded